

UNIVERSIDAD INTERNACIONAL DEL ECUADOR

Programación Orientada a Objetos 2



LeeLibre

Sistema de Gestión de Libros Electrónicos

Proyecto Integrador — Informe Final

Alumno: Cortez Villa Ronny

Docente: Ing. Torres Arévalo Carlos

Entrega Final — Integración de las Cuatro Unidades

23 de agosto de 2026

Resumen

Este informe cierra el proyecto integrador de la asignatura y describe LeeLibre, un sistema de gestión de biblioteca digital construido en Go a lo largo de ocho semanas. El sistema permite consultar un catálogo, registrar préstamos con vencimiento, gestionar una cola de espera cuando un título no está disponible, y obtener recomendaciones de lectura personalizadas. Sobre un mismo núcleo de dominio se levantan dos interfaces independientes: una aplicación de consola y un servidor de trece servicios web REST con serialización JSON, protegido mediante un middleware de autenticación. El resultado son unas 3.500 líneas de código Go distribuidas en diez paquetes, sin ninguna dependencia externa, acompañadas de 48 pruebas automatizadas que se ejecutan con el detector de condiciones de carrera del lenguaje. El documento recorre las decisiones de diseño tomadas en cada unidad, justifica los cambios respecto de la planeación inicial, detalla las pruebas realizadas y sus resultados, y expone tanto los alcances como las limitaciones conscientes del trabajo.

Palabras clave: programación orientada a objetos, Go, encapsulación, interfaces, concurrencia, servicios web REST, pruebas automatizadas.

Introducción

Un proyecto que se construye por etapas tiene una ventaja sobre uno que se escribe de una sola vez: obliga a convivir con las decisiones tomadas antes. Lo que en la primera semana parecía razonable tuvo que sostenerse, corregirse o abandonarse conforme el sistema fue creciendo, y ese recorrido es tan parte del resultado como el código final.

LeeLibre nació como una planeación sin código, se convirtió en un dominio orientado a objetos durante las unidades dos y tres, y terminó siendo un servicio web concurrente con pruebas automatizadas. Este informe recorre esa evolución en orden: primero el problema y por qué se eligió, después las decisiones de diseño agrupadas por unidad, luego la verificación realizada, y finalmente lo que quedó fuera y por qué.

Conviene anticipar dos decisiones que atraviesan todo el trabajo. La primera es que no se utilizó ningún paquete de terceros: el sistema completo compila con la biblioteca estándar de Go. La segunda es que la planeación inicial proponía programación funcional y se corrigió hacia la programación orientada a objetos. Ambas se justifican más adelante.

Justificación del tema

El enunciado ofrecía tres sistemas de gestión posibles y dejaba abierta la opción de proponer otro. Se eligió la biblioteca digital por tres razones concretas.

La primera es que el dominio tiene reglas propias que no son un simple registro de datos. Un libro prestado no puede volver a prestarse; un préstamo tiene un plazo que vence solo, sin que nadie lo toque; un usuario que llega tarde a un título ocupado tiene que esperar su turno. Son reglas que obligan a que los objetos protejan su estado en lugar de limitarse a guardarlo, que es justamente lo que la asignatura pide demostrar.

La segunda es que ese mismo dominio ofrece un lugar natural para ir más allá del alta, baja y consulta. El historial de préstamos no es solo un registro: es información sobre qué lee cada persona, y a partir de ahí se puede construir algo que el sistema no tenía obligación

de hacer. De ahí salió el recomendador.

La tercera es personal. Una biblioteca es un sistema que uno entiende sin necesidad de que se lo expliquen, y eso permitió concentrarse en cómo modelarlo en lugar de en averiguar cómo funciona el negocio.

Aplicaciones prácticas

El modelo no se agota en una biblioteca. La estructura de fondo —un catálogo de recursos limitados, usuarios con distintos permisos, préstamos con vencimiento y una cola de espera cuando el recurso está ocupado— se repite en el préstamo de equipos de laboratorio, en la reserva de aulas o salas, en el alquiler de herramientas y en la asignación de licencias de software por puestos. Cambian los nombres de las entidades; las reglas son las mismas.

Descripción del sistema

Qué hace

El catálogo permite consultar todos los libros, buscar por título o autor sin distinguir mayúsculas, y filtrar por categoría o por disponibilidad. El administrador puede además agregar, modificar y retirar títulos, con validaciones que impiden guardar un libro sin título, con un año fuera del rango 1000 a 2100 o con un formato distinto de PDF o EPUB.

Los usuarios tienen uno de dos roles, lector o administrador, y sus permisos difieren. La contraseña se puede verificar pero no leer: es el único campo del sistema sin método de acceso.

Un préstamo se otorga por catorce días y calcula su fecha de vencimiento al crearse. Pasa por tres estados —vigente, devuelto y vencido— y solo puede devolverlo su titular o un administrador.

Cuando un lector pide un libro que otro tiene, el sistema no se limita a rechazar la petición: le ofrece entrar en una cola de espera. Al devolverse el título, el primero de la fila queda notificado.

Los reportes incluyen el ranking de libros más prestados y el recomendador personalizado, y pueden generarse varios a la vez.

Arquitectura

```
leelibre/
|-- cmd/
|   |-- leelibre/main.go      aplicación de consola
|   |-- server/main.go       servidor REST y carga concurrente
|-- internal/
|   |-- errores/              errores centinela del dominio
|   |-- catalogo/            libros, categorías y catálogo
|   |-- usuarios/            usuarios, roles y autenticación
|   |-- prestamos/           préstamos e historial
|   |-- reservas/            cola FIFO de espera
|   |-- reportes/            interface Reportador y ejecución concurrente
|   |-- persistencia/        guardado en JSON con escritura atómica
|   |-- api/                  handlers HTTP, DTO y middleware
|-- data/                     datos de ejemplo en JSON
|-- docs/                      documentación y diagramas de las tres etapas
```

La dirección de las dependencias fue un criterio sostenido durante todo el desarrollo. Los paquetes del dominio no conocen la capa web; el paquete `catalogo`, que es el más básico, no depende de `usuarios` ni de `prestamos`; y `reportes` es el único que consulta a varios, lo cual es coherente con su función de agregar información. Gracias a esa disposición, los dos ejecutables consumen el mismo núcleo y la lógica de negocio no está duplicada en ninguna parte.

Diagrama de clases

El diagrama completo, en formato editable, está en `docs/diagrams/`. La tabla siguiente resume las relaciones que representa.

Tabla 1

Relaciones entre los tipos del sistema.

Tipo	Relación	Con	Significado
Catalogo	composición	Libro, Categoría	administra su ciclo de vida
Libro	asociación	Categoría	pertenece a una
Autenticador	composición	Usuario	administra el registro
Historial	composición	Prestamo	administra los préstamos
GestorReservas	composición	Cola	una cola por libro
MasPrestados	implementa	Reportador	satisface la interface
Recomendador	implementa	Reportador	satisface la interface
Server	dependencia	todos los anteriores	los consulta, no los posee

Decisiones de Diseño por Unidad

Unidad 1: fundamentos y organización en paquetes

La primera decisión de peso fue separar el código en paquetes por responsabilidad en lugar de acumular archivos en uno solo. En Go la visibilidad se resuelve a nivel de paquete: lo que empieza con minúscula no sale de él. Esa regla, que parece una convención de nombres, es en realidad la herramienta de encapsulación del lenguaje, y aprovecharla exigía decidir desde el principio qué pertenecía a cada paquete.

Unidad 2: estructuras de datos

Cada estructura se eligió por el uso que se le iba a dar. Los *slices* sostienen las colecciones que crecen y se recorren enteras: el catálogo, el historial, la cola. Los *maps* resuelven los accesos directos por clave: el índice de usuarios por correo para el inicio de sesión, el perfil del lector por categoría, el conteo de préstamos por libro.

No se utilizó ningún *array* de tamaño fijo, y la ausencia es deliberada. Ninguna colección del dominio tiene un tamaño conocido de antemano: el catálogo crece cuando

se agregan libros y la cola de reservas depende de cuánta gente esté esperando. Declarar un array impondría un límite artificial que después habría que justificar. Cuando la estructura correcta es el slice, forzar un array para cumplir con una lista de temas empeora el diseño.

La cola FIFO del paquete `reservas` merece una mención aparte porque no se eligió por conveniencia técnica sino porque el dominio la pedía: cuando varias personas esperan el mismo libro, el orden de llegada es lo que decide quién lo recibe primero. Internamente se apoya en un slice, pero su interfaz pública es la de una cola pura.

Unidad 3: encapsulación, errores e interfaces

Todos los campos de las entidades son privados y se leen mediante captadores que, siguiendo la convención de Go, no llevan el prefijo `Get`: el captador de `titulo` se llama `Titulo`.

Los modificadores son la justificación práctica de esa decisión. Un campo privado solo tiene sentido si el método que lo cambia puede negarse:

```
func (l *Libro) SetAnio(nuevo int) error {
    if nuevo < 1000 || nuevo > 2100 {
        return fmt.Errorf("%w: el año debe estar entre 1000 y 2100, se recibió %d",
            errores.ErrDatosInvalidos, nuevo)
    }
    l.mu.Lock()
    defer l.mu.Unlock()
    l.anio = nuevo
    return nil
}
```

Si el campo fuera público, cualquier parte del programa podría dejar el libro con el año 3500 y nada lo advertiría. Los modificadores aplican las mismas reglas que el constructor, de modo que un libro creado y uno modificado cumplen siempre las mismas condiciones.

Los errores del dominio están centralizados como valores centinela y se comparan con `errors.Is`, no comparando textos. Cuando hace falta añadir contexto se envuelven con el

verbo `%w`, que conserva la comparabilidad del error original.

El polimorfismo se concentra en la interface `Reportador`, con dos implementaciones que se usan de manera uniforme sin que el código llamador sepa cuál tiene entre manos. Aparece también de forma implícita en `fmt.Stringer`: al definir el método `String` en `Libro` y `Prestamo`, cualquier función de impresión los muestra ya formateados sin declarar en ningún sitio que se implementa esa interface.

Unidad 4: concurrencia, servicios web y pruebas

Exponer el sistema como servidor introdujo una clase de problema que la consola no tenía. El servidor de la biblioteca estándar atiende cada petición en una *goroutine* distinta, de modo que varias pueden tocar el catálogo al mismo tiempo, y ni los slices ni los maps de Go admiten ese uso. El detector de carreras del lenguaje confirmó doce accesos conflictivos al someter al servidor a peticiones simultáneas.

La corrección consistió en proteger las colecciones y las dos entidades cuyo estado cambia con `sync.RWMutex`: las consultas toman el candado en lectura, que admite varios lectores a la vez, y las modificaciones lo toman en escritura, que es exclusivo. Dos detalles importaron. El primero es que la comprobación y la escritura ocurren bajo un mismo bloqueo, porque hacerlas por separado permitiría que dos peticiones comprobaran que un libro está disponible y lo prestaran ambas. El segundo es que `sync.RWMutex` no es reentrante, así que los métodos que se apoyan en otros del mismo tipo usan una versión interna sin bloqueo.

La concurrencia no aparece solo como protección. Los reportes se generan en paralelo porque son independientes entre sí, y el recomendador —el más costoso, ya que recorre el historial completo y después el catálogo— no tiene por qué bloquear a los demás:

```
for i, r := range reportes {
    wg.Add(1)
    go func(orden int, rep Reportador) {
        defer wg.Done()
        canal <- ResultadoReporte{Orden: orden, Nombre: rep.Nombre(),
```



```
        Contenido: rep.Generar() }  
    }(i, r)  
}  
wg.Wait()  
close(canal)
```

El canal se crea con búfer porque los resultados se leen después del `Wait`: sin búfer, las goroutines quedarían bloqueadas al escribir y el programa no avanzaría. Y como terminan en orden impredecible, cada resultado viaja con su posición original y al final se reordena; de lo contrario, dos ejecuciones seguidas devolverían los reportes en distinto orden.

El servidor usa además `sync/atomic` para contar las peticiones atendidas. Un entero normal no sirve: dos peticiones simultáneas harían leer, sumar y escribir a la vez, y una de las dos sumas se perdería. Aquí basta con una operación atómica y no hace falta un mutex, porque lo único que se hace con el contador es sumarle uno.

Servicios Web

El servidor expone trece rutas construidas sobre `net/http`, todas con respuesta en JSON.

Tabla 2

Servicios web expuestos y nivel de acceso.

Método	Ruta	Acceso
GET	/	Público
GET	/api/libros	Público
GET	/api/libros/{id}	Público
POST	/api/libros	Administrador
PUT	/api/libros/{id}	Administrador
DELETE	/api/libros/{id}	Administrador
GET	/api/categorias	Público
POST	/api/prestamos	Autenticado
POST	/api/prestamos/{id}/devolver	Titular o administrador
GET	/api/usuarios/{id}/recomendaciones	Público
GET	/api/reportes/mas-prestados	Público
GET	/api/reportes/resumen	Público
GET	/api/reservas/{libro_id}	Público

Serialización

Las respuestas se construyen con objetos de transferencia propios de la capa web, anotados con etiquetas de campo. Esa separación evita dos problemas: exponer los tipos del dominio hacia fuera y tener que contaminarlos con etiquetas de serialización que no les corresponden. Hay además un motivo técnico: `encoding/json` solo serializa campos exportados, y los del dominio son privados por diseño, de modo que serializarlos directamente produciría objetos vacíos.

Autenticación y autorización

Las operaciones que modifican datos exigen credenciales, resueltas con autenticación básica de HTTP mediante `r.BasicAuth` de la biblioteca estándar.

El control se implementó como *middleware*: una función que recibe un handler y devuelve otro con comportamiento añadido alrededor. `conAuth` exige credenciales válidas y entrega el usuario ya identificado al handler; `conAdmin` se construye sobre la anterior y añade la comprobación del rol. Así la verificación no se repite en cada handler y resulta imposible

olvidarla al agregar uno nuevo.

Esta construcción distingue tres preguntas que suelen confundirse: quién realiza la petición —si no puede establecerse, la respuesta es 401—, si tiene permiso para la operación —si no, 403— y si puede actuar sobre ese recurso concreto. La tercera es la que más se olvida: devolver un préstamo exige ser su titular, porque estar autenticado no basta para operar sobre algo ajeno.

Una decisión relacionada afecta al registro de préstamos. En una versión anterior el identificador del usuario viajaba en el cuerpo de la petición, lo que permitía pedir libros a nombre de otra persona. Ahora el préstamo se registra siempre a nombre de quien se autentica y el cuerpo solo indica qué libro se solicita: la identidad del solicitante no puede provenir del cliente.

Pruebas Realizadas y Resultados

El sistema se verificó de tres formas complementarias.

Análisis estático

La compilación completa no produce advertencias, el analizador `go vet` no reporta observaciones y `gofmt` no encuentra desviaciones respecto del estilo oficial del lenguaje.

Pruebas automatizadas

Se escribieron 48 funciones de prueba repartidas en seis paquetes, con el paquete `testing` de la biblioteca estándar. Las validaciones y las reglas de estado usan pruebas basadas en tabla: cada caso es una fila de un slice y se ejecuta con `t.Run`, de modo que la salida indica exactamente cuál falló y agregar un caso nuevo consiste en agregar una línea.

Los endpoints se prueban con `httptest.NewRecorder`, que implementa `http.ResponseWriter` en memoria: se verifica el código de estado y el cuerpo de la respuesta sin abrir ningún puerto ni levantar un servidor real.

Tabla 3

Cobertura de pruebas por paquete.

Paquete	Cobertura	Foco principal
internal/reservas	94,1 %	orden FIFO y concurrencia
internal/catalogo	65,8 %	validaciones y filtrado
internal/api	60,4 %	permisos y serialización
internal/prestamos	54,7 %	estados y vencimiento
internal/usuarios	51,0 %	autenticación

Más allá del porcentaje, lo que se decidió cubrir fueron las reglas que importan: que un libro prestado no pueda prestarse otra vez, que un préstamo venza a los catorce días, que los modificadores rechacen valores imposibles y el objeto conserve su valor anterior en lugar de quedar a medio cambiar, que un usuario no pueda pedir libros a nombre de otro ni devolver un préstamo ajeno, y que los métodos que devuelven colecciones entreguen una copia para que nadie pueda alterar el estado interno desde fuera.

Probar el vencimiento exigió un cambio en el diseño. El método que actualiza el estado consultaba el reloj del sistema, y con eso no había forma de comprobar la regla salvo esperar catorce días o cambiar la hora del computador. Ahora recibe la fecha como parámetro, y la prueba le pasa una fecha futura y verifica el resultado al instante. Es un caso claro de cómo escribir pruebas mejora el diseño en lugar de limitarse a comprobarlo.

Verificación de concurrencia

Toda la batería se ejecuta con `go test -race`, que detecta accesos concurrentes conflictivos. Antes de introducir los candados, la prueba de carga reportaba doce; después de corregirlos, ninguno.

Dos pruebas atacan la concurrencia directamente. Una lanza cincuenta reservas simultáneas y comprueba que no se pierde ninguna, propiedad que no depende del orden en que lleguen. La otra mide que los reportes en paralelo tarden menos que en secuencia: tres reportes de 60 milisegundos cada uno tardaron 61 milisegundos en total, frente a los 180 que

habrían tardado uno detrás de otro.

Visualización del Futuro

Las tecnologías elegidas condicionan hacia dónde puede crecer el sistema, y en este caso lo habilitan más de lo que lo limitan.

El paso inmediato es la interfaz web visual. Al haber separado el dominio de la capa de presentación desde el principio, y al entregar ya todo en JSON, agregar un cliente web no exige tocar la lógica de negocio: consume los mismos endpoints que hoy responden a `curl`. La misma propiedad abre la puerta a una aplicación móvil sin duplicar nada del servidor.

El segundo paso natural es sustituir los archivos JSON por una base de datos. La capa de persistencia está aislada tras funciones propias, de modo que el cambio afectaría a un paquete y no al resto del sistema. Con datos persistentes y consultas reales, los reportes podrían ir más allá del conteo: qué categorías crecen en demanda, en qué momentos del año se presta más, qué títulos llevan meses sin salir del estante.

El tercero es el que la concurrencia hace posible. Hoy se usa para generar reportes en paralelo y proteger el estado compartido, pero el mismo mecanismo sostiene tareas de fondo: revisar periódicamente los préstamos vencidos y avisar por correo, procesar la cola de reservas cuando un libro se libera, o recalcular recomendaciones sin bloquear al usuario que está navegando.

Más a futuro, el historial de lecturas es el activo que el sistema acumula sin proponérselo. El recomendador actual funciona con una regla simple —la categoría más leída—, pero los mismos datos admiten técnicas más finas: sugerir a partir de lo que leyeron personas con perfiles parecidos, o detectar que alguien está cambiando de intereses antes de que lo note. Ese es el punto donde un sistema de gestión deja de limitarse a registrar lo que pasa y empieza a aportar algo sobre ello.

Alcances y Limitaciones

Lo que el sistema hace

Gestión completa del catálogo con validaciones; usuarios con dos roles y permisos diferenciados; préstamos con plazo, estados y control de titularidad; cola de reservas con orden garantizado; recomendador personalizado con justificación de cada sugerencia; trece servicios web con autenticación y autorización por rol y por recurso; persistencia en JSON con escritura atómica; ejecución concurrente de reportes; y una batería de 48 pruebas automatizadas.

Lo que no hace, y por qué

No tiene interfaz web visual. El sistema entrega JSON; construir la pantalla que lo consume corresponde a programación web y excede el alcance de esta asignatura. La arquitectura ya está preparada para ello.

No usa base de datos. Para el volumen de datos previsto la diferencia de rendimiento es irrelevante, y los archivos JSON permiten inspeccionar el estado del sistema con un editor de texto, lo cual resultó útil durante el desarrollo.

Las contraseñas se guardan en texto plano. Son datos de ejemplo ficticios. En un sistema real irían cifradas con `bcrypt`; se declara como limitación consciente y no como olvido, porque la planeación inicial sí lo contemplaba.

La cobertura de pruebas no es total. Se priorizaron las reglas de negocio y la seguridad sobre el porcentaje. Los paquetes de persistencia y los ejecutables no tienen pruebas propias.

No hay control de stock por ejemplares. Cada libro es una unidad: o está disponible o no. Un sistema real manejaría varias copias del mismo título.

Conclusiones

El proyecto cumplió su objetivo: el sistema planificado en la primera semana existe, se ejecuta, está probado y puede demostrarse. De las ocho semanas de trabajo quedan cuatro aprendizajes que exceden a este código concreto.

El primero es que la encapsulación solo adquiere sentido cuando aparecen los modificadores. Mientras hay únicamente captadores, hacer privados los campos parece una formalidad que solo añade líneas. Cuando un modificador rechaza un año imposible y devuelve un error, la razón de esa decisión queda a la vista: el objeto se defiende de quien lo usa mal.

El segundo es que exponer un sistema como servidor cambia su naturaleza. Ni el compilador ni el analizador estático advierten de un acceso concurrente conflictivo; hizo falta ejecutar el programa bajo el detector de carreras para encontrar doce. Leer el código no sustituye a ejecutarlo bajo las condiciones reales en que va a funcionar.

El tercero es que añadir autenticación no resuelve por sí solo el problema de la suplantación. Mientras el identificador del usuario siguió llegando en el cuerpo de la petición, cualquiera podía actuar en nombre de otro pese a estar correctamente autenticado. La identidad de quien realiza una acción debe provenir de la sesión y nunca del contenido que envía el cliente.

El cuarto apareció al escribir las pruebas, y fue el más inesperado. Un método que consultaba el reloj del sistema resultaba imposible de probar, y la solución no fue escribir una prueba más elaborada sino cambiar el método para que recibiera la fecha. Las pruebas no solo comprueban el diseño: lo empujan a mejorar.

Queda también una reflexión sobre las dificultades. La mayor no fue técnica sino de criterio: decidir cuándo un tema de la asignatura correspondía al problema y cuándo se estaría forzando. Los arrays son el ejemplo claro. Se podría haber declarado uno para tenerlo en la lista, pero ninguna colección del dominio tiene tamaño fijo, y forzarlo habría empeorado el diseño para cumplir un requisito de forma. Se optó por dejarlo fuera y explicar el motivo, que es lo que se ha hecho en este informe.

Referencias

- 1 American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7a ed.). <https://doi.org/10.1037/0000165-000>
- 2 Donovan, A. A. A., & Kernighan, B. W. (2015). *The Go programming language*. Addison-Wesley.
- 3 Sommerville, I. (2016). *Software engineering* (10a ed.). Pearson.
- 4 Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2a ed.). Addison-Wesley.
- 5 Meyer, B. (1997). *Object-oriented software construction* (2a ed.). Prentice Hall.
- 6 The Go Authors. (2024). *Effective Go*. https://go.dev/doc/effective_go
- 7 The Go Authors. (2024). *The Go memory model*. <https://go.dev/ref/mem>
- 8 Fielding, R. T., & Reschke, J. (2014). *Hypertext transfer protocol (HTTP/1.1): Semantics and content* (RFC 7231). Internet Engineering Task Force. <https://doi.org/10.17487/RFC7231>
- 9 Beck, K. (2002). *Test-driven development: By example*. Addison-Wesley.